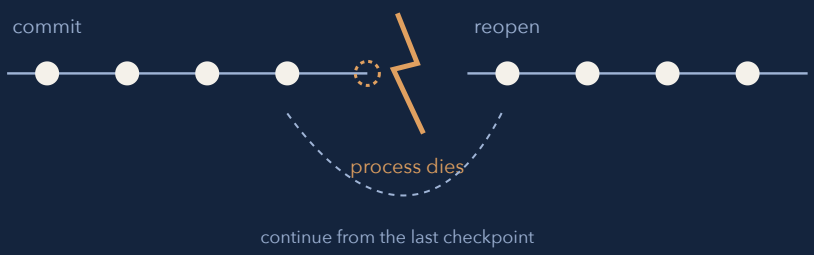
EARENDIL · FIELD GUIDE

Pi Durable

Agents that outlive their process. The why, the architecture, and the story of a crash, explained without code.



Background reading for a ten-minute talk on `@earendil-works/pi-durable`
Based on `packages/durable` in pi-mono (v1.2.0, October 2026),
its specification, its git history, and the launch post “Pi Durable” (1 October 2026).

Contents

BEFORE YOU START

How to read this book

PART I · WHY

1 The agent that lives in a terminal
2 Agents leave the terminal
3 The road to Pi Durable
4 A vocabulary for harnesses

PART II · HOW IT WORKS

5 The one rule: commit before you show
6 What lives in storage
7 Tasks: work as durable state machines
8 One turn, taken apart
9 One ownership tree
10 Many conversations: forks and subagents
11 The inbox: many people, one agent
12 Context without end
13 Extensions: names, not code
14 Documents, views and multiplayer
15 Running anywhere
16 What can go wrong, and what is not promised

PART III · SEEING IT WORK

17 Pulling the plug: a crash, commit by commit
18 The vacation planner, underneath
19 Not just for models: a split payment

PART IV · TOWARD THE TALK

20 How to demo something you should not demo live
21 A ten-minute shape
22 Facts, numbers, and things to double-check
A Glossary
B A map of the source

BEFORE YOU START

How to read this book

This book exists to make a ten-minute talk possible. Ten minutes is too short to teach Pi Durable. It is long enough to leave people with one picture in their heads, and to give that picture you have to understand the whole thing first. The book tries to get you there.

It deliberately avoids code. Where code appears, it shows a shape, never an API to memorize. The goal is that you can explain Pi Durable on a whiteboard: what is stored, who does the work, what happens when the power goes out.

- **Part I** explains why Pi Durable exists: the assumptions a terminal coding agent can make, why they stop holding once agents leave the terminal, and how the design arrived where it is.
- **Part II** is the architecture, in the order you would explain it: the one rule everything follows, what lives in storage, tasks, a turn taken apart, ownership, and then the features that fall out of those.
- **Part III** walks through three scenarios in slow motion. These are the raw material for a demo that explains instead of just showing.
- **Part IV** turns all of it into options for the talk: demo formats, a ten-minute structure, and a list of facts to double-check before going on stage.

If you only have half an hour: read chapters 2, 5, 7, 9 and 17. Those are the spine of the talk.

SOURCES

The normative specification in `packages/durable/docs/spec.md` (about 5,300 lines), the package README, the examples in `test/examples`, the git history of `packages/agent` and `packages/durable`, and the launch post on earendil.com. Where this book interprets rather than reports, it says so.

PART I

Why

A coding agent in a terminal can lean on a human for recovery. An agent in Slack, in an inbox, or in a Durable Object cannot. Pi Durable is what you build when no human is watching the process.

CHAPTER 1

The agent that lives in a terminal

Pi, the coding agent, runs on your machine (or a remote one), inside a terminal, driven by one person. It is minimal by design: a short system prompt, four tools, and an extension system that lets people mold it into what they need. More than five thousand extensions exist because of that malleability.

Pi stores a session as a transcript: a tree of entries, one JSON object per line, on disk. The process runs the agent loop: send the transcript to the model, run the tool calls it asks for, append the results, repeat until the model answers. The interesting state while this happens lives in memory: the half-streamed answer, the output of a running `bash`, the message you typed while the agent was busy, the retry it is about to attempt.

When the process dies, all of that is gone, and that is fine. You look at the transcript, see where it stopped, and type "continue". **The human is the recovery mechanism.** Pi 1.0 is built around this, excels at it, and is not going to change.

The assumptions a terminal agent gets to make

It is worth spelling out what this design quietly relies on, because each of these is something Pi Durable had to give up:

Assumption	Why it holds in a terminal
One process, one session, one human	You started it, you are watching it, nobody else types into it.
The conversation lives as long as the process	Sessions last minutes or hours; you close the laptop, you restart.
In-flight state can be volatile	If a tool call is lost, you will notice and redo it.
The transcript is the only state	There is no application around the agent that needs its own durable state.
The UI is in the same process	Events flow from the loop to the screen in memory.
The code is fixed while it runs	To change an extension you reload; a run in progress is your problem.

None of these are flaws. They are what makes a terminal agent small. But every one of them breaks the moment the agent is not a program you are personally watching.

CHAPTER 2

Agents leave the terminal

Earendil wants this technology in front of everyone, in whatever form fits them: Lefos answers by email, a bot answers mentions in Slack, a triage bot labels GitHub issues, an agent runs inside a Cloudflare Durable Object behind a web page. None of these has a human watching the process.

The launch post names five requirements. Each one breaks one of the assumptions from the last chapter.

Requirement	Breaks the assumption that…	What it forces
Runs anywhere	there is one machine with a file system and Node	Storage and execution behind small interfaces; no Node APIs in the core.
Reachable from different surfaces	the UI lives in the process	UIs attach to committed state, from anywhere, at any time.
Infinitely long conversations	a conversation fits in a process lifetime and in a context window	Bounded memory, compaction while working, nothing ever deleted.
Survives catastrophic failures	in-flight state can be volatile	Every step checkpointed; a new process continues the old one's work.
Multiple humans steer the same agents	one person types into it	Durable queues, steering, exactly-once submission.

There is a sixth requirement that is easy to miss: **malleability**. Pi's soul is that you change it. An agent that runs for weeks still needs its code changed, and you cannot stop the world to do it.

Who recovers when nobody is watching?

This is the core of the "why". In the terminal, a crash is an inconvenience. In a Slack bot, a crash in the middle of a tool call means a question that is never answered, a deploy that may or may not have happened, and a user who asked twice because the bot looked dead. When the container is redeployed, the laptop sleeps, or the machine runs out of memory, someone has to know what was in progress and what to do about it.

If the human is no longer the recovery mechanism, **the harness has to be**. That single sentence is the reason Pi Durable exists.

Why not change Pi the coding agent?

Because Pi works and people depend on it. Pi Durable is a separate, experimental package. It shares code with the coding agent (model access through `pi-ai`) and its principles (minimalism and malleability), but it is a framework for building *any* agentic application, coding agents included. It is also a place to try designs without disrupting Pi: lessons that prove themselves are meant to flow back.

FOR THE TALK

"A coding agent can rely on you to notice that it died. A Slack bot can't. Pi Durable is what you build when nobody is watching the process."

CHAPTER 3

The road to Pi Durable

Pi Durable did not start as a database of tasks. It started as a much more modest idea: keep the session file, and add recovery notes to it. The git history shows the design moving, over about ten weeks, from one to the other.

A timeline from the repository

When (2026)	What happened
28 July	A first "Durable AgentHarness" design document lands in `packages/agent`: about 2,100 lines. It keeps Pi's session log and adds private *harness entries* to it.
August	Harness v2 and v3 implementations, designs for plugins and reload, sessions/branches/lanes, a "durable drive" runtime. Many design documents, many revisions.
Early September	A series of prototypes named *Pico*: a pico2 spike, a hardened pico3 kernel, then Pico5, built on Chord's transactional replicated state.
18 September	Pico moves into its own package: `packages/durable`. Over the next two weeks: SQLite and JSONL storage, transactional sessions, documents, forks, the task runtime, the Harness, structured concurrency.
1 October	Shipped as experimental alongside Pi 1.0, together with a small coding agent and a vacation planner built on it.
Since	Durable code mode, nested tool calls, tool search, structured output, a stricter error model. About 150 commits touch the package.

Version one: a transcript with recovery notes

The July design is a natural first idea. Pi already has an append-only session log. So: write extra, private records into the same log as you go: "operation started", "tool started", "message queued". They are never shown to the model and never part of the tree. After a crash, a recovery procedure reads the end of the log, works out how far the run got, and continues. Its central invariant reads:

> *Session entries define what the conversation is. Harness entries define what the harness did, in what order.*

It also had to say what it could not do: at most one operation per session at a time, hooks never re-run after a crash, provider streams never persisted.

Where it ended up: a database of records

The specification that ships today opens with a different core rule:

> *A Session atomically commits immutable entries, full task records, and Chord-tracked documents. Only committed state is observable.*

Read the two side by side and you can see the shift. (This is an interpretation, but the documents support it.)

	July: log plus recovery notes	October: Pi Durable
Source of truth	The transcript, with orchestration facts beside it	Conversations, entries, tasks, submissions and documents, all committed together
Recovery	A procedure that reads the log and reconstructs	Not special: running tasks become pending, the scheduler continues
Concurrency	One operation per session	Any number of conversations and tasks, in one ownership tree
In-flight state	Volatile; partial streams never persisted	Committed: partials and tool output are durable documents
UI model	Snapshot plus live events	Snapshot plus exact operations of each commit
Application state	Not addressed	Typed documents, committed with the transcript

The lesson is the most interesting thing to say about the history in a talk: **recovery stopped being a feature and became a consequence**. If every unit of work is a durable state machine and nothing is visible until it is stored, there is nothing to reconstruct. A new process simply has the same state the old one had.

FOR THE TALK

"Our first design was a transcript with recovery notes in the margin. The design we shipped is a small database where the transcript is just one table, and every piece of work is a row that knows where it was."

CHAPTER 4

A vocabulary for harnesses

Everybody has their own definition of "harness". Earendil's earlier post compared it to a climbing harness: it holds you safely, connects you to ropes and tools, and you adapt it to the climb. Pi Durable needs a sharper definition, because it is a harness for building harnesses.

> *A harness is storage plus the machinery needed to run one or more conversations with large language models in parallel. It provides the tools those models call, and the execution environments the tools run in.*

Six words carry the rest of this book:

Word	Meaning
Conversation	An interaction between you and an agent, recorded as a transcript of immutable entries.
Agent	The model together with its settings (thinking level, instructions) and the tools it can call. Stored per conversation.
Tool	Something the model can call. Each call is durable work of its own.
Execution environment	Where tools touch files and processes: your laptop, a remote VM, a container, an in-memory sandbox.
Task	Everything the harness runs, from calling the model to executing a tool, is a task: a durable state machine.
Extension	A named bundle of prompt sections, tools, hooks and tasks. Conversations select extensions by name.

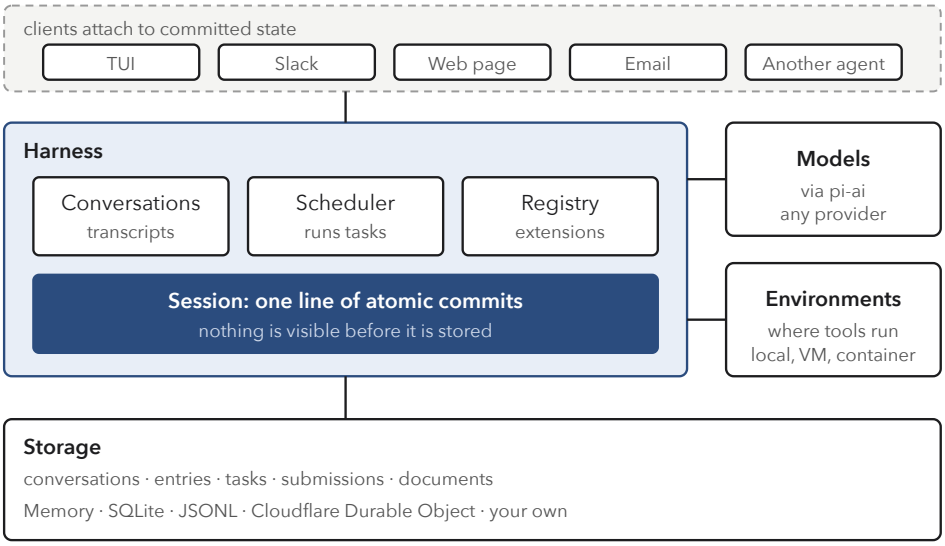


Figure 4.1 · The pieces. Everything between the clients and the storage is replaceable; the only fixed point is that all state goes through one line of commits.

Like everything in Pi, Pi Durable is meant to be understood by your agent. At launch the source, without tests, was about 15,000 lines (about 150,000 tokens for GPT, 250,000 for Claude) and it has grown since. An agent building on it rarely needs all of it; the storage backends alone are 3,000 lines it can usually skip.

PART II

How it works

One rule, five kinds of records, and one kind of worker. Almost every feature of Pi Durable falls out of these.

CHAPTER 5

The one rule: commit before you show

If you remember one thing about Pi Durable, remember this: nothing is visible until it is stored. Not a streamed token, not a line of tool output, not a queued message. Everything else is built on that rule.

The mutation line

A harness opens over one storage. Inside it, a **Session** owns a single *mutation line*: a serialized queue through which every change passes. A change is a **commit**: a function that, in one transaction, may append transcript entries, change documents, create tasks, and record submissions. Either all of it is stored, or none of it is.

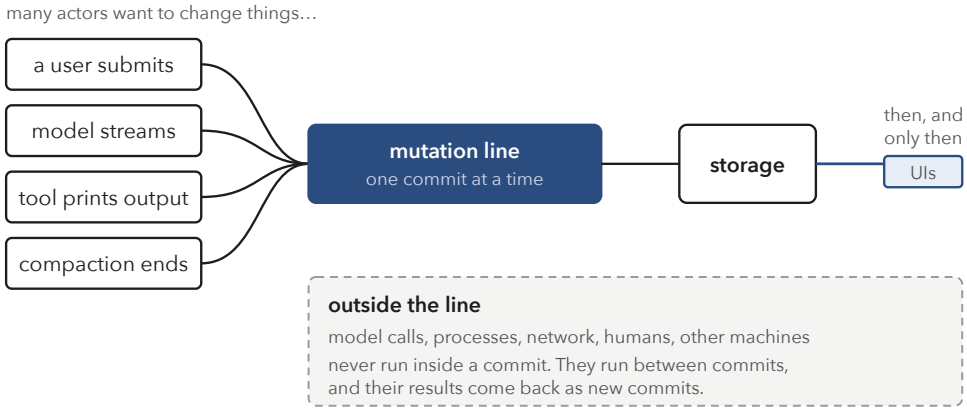


Figure 5.1 · All changes pass through one line. Effects happen outside it; observers see only what storage accepted.

Two consequences follow immediately:

- **Effects never run inside a transaction.** A commit cannot wait on a model, a process or a human. This keeps commits short and makes the boundaries between “decided” and “done” explicit.
- **There is no volatile path to the screen.** A streaming answer is committed as a partial at most every 100 milliseconds. A tool’s running output is committed the

same way. A UI sees them because they are in storage, not because the process told it.

Why this one rule buys so much

Property	Why it follows
A crash loses almost nothing	Whatever anyone saw was stored. At most the last throttle window of streaming (100 ms by default) is gone.
Recovery is just reopening	The new process reads the same state the old one was showing. There is no second copy to reconcile.
Any number of viewers	A UI is a reader of committed state. Ten UIs, or one that joins late, all read the same thing.
Consistency between parts	A tool's result entry and the todo list it updated are in the same commit. They can never disagree.
Memory is a cache	The process keeps a working set in memory. Anything not in use can be dropped and reloaded.

AN ANALOGY

The screen is a window onto the database, not onto the process. When the process dies, the window does not go dark: it just stops changing until a new process takes over the same database.

THE PRICE

Every visible change costs a commit. Partials are throttled so this stays cheap on local SQLite; a host with remote storage can commit less often (say every 500 ms). And one process owns a storage at a time: there is no cross-process locking. Other clients attach to that process.

CHAPTER 6

What lives in storage

Storage holds four tables of records and one kind of mutable state. That is the entire durable model. If you can draw this page, you understand what survives a crash.

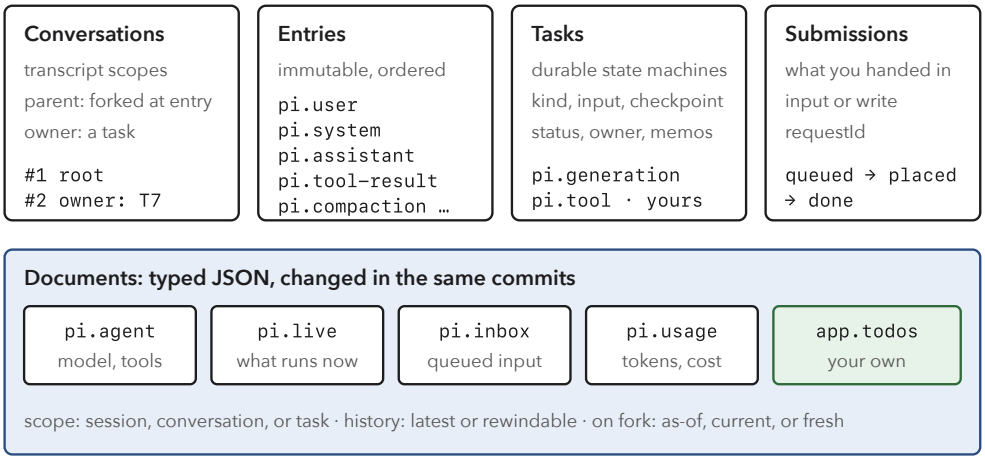


Figure 6.1 · The durable model. One commit can touch any of these at once.

The four tables

Conversations are transcript scopes. A conversation has two optional edges: a *parent* (it was forked from that conversation at a given entry) and an *owner* (a task created it, as a subagent). These two edges are deliberately separate: one is about history, the other about responsibility.

Entries are the transcript. They are immutable, their IDs are ordered across the whole session and never reused. Built-in kinds are user messages, model answers, tool results, system prompt changes, resets and compaction summaries. Applications add their own kinds, for example a chat mention that shows who asked.

Tasks are units of work with a stored checkpoint. They are the subject of the next chapter.

Submissions are what someone handed to a conversation: user input (which asks the model something) or a write (an entry to append without asking anything). A submission moves from *queued* to *placed* to *done*, or ends *unanswered* with a reason. An optional `requestId` makes it exactly-once: submit the same request ID again, after a crash or a network retry, and you get the original submission back.

Documents: the part that changes

Entries never change, but some things obviously do: the model selected for a conversation, the output of the tool that is running right now, the queue of messages typed while the agent was busy, the money spent. These live in **documents**: typed JSON values stored next to the transcript and changed in the same commits. Under the hood, a document change is recorded as a small list of JSON operations (via Chord, Earendil's replicated-state library), with an occasional full copy so that replay stays short.

The built-in documents are worth knowing by name, because they are what makes "what is happening right now" durable:

Document	Holds
`pi.agent`	What this conversation runs with: model, thinking level, selected extensions and tools (as names), instructions, working directory.
`pi.live`	The run in progress: the streaming partial answer, retry backoff, every running tool call with its output so far, nested calls, compactions in flight.
`pi.inbox`	Submissions waiting for a turn boundary: steers, follow-ups, writes.
`pi.usage`	Tokens and cost, per model and per tool, including failed and aborted attempts.
`pi.provider`	A stable ID given to the model provider, so prompt caches keep working across restarts.

Documents also have a lifetime: *session* documents live forever, *conversation* documents belong to a conversation, and *task* documents disappear when their task finishes. And conversation documents declare what a fork sees: the value at the fork point, the current value, or a fresh one. That is a product decision, not an optimization (chapter 14).

The storage itself

The storage interface is small. Pi Durable ships four backends and a conformance suite plus benchmarks for writing your own (a key-value store or Postgres would do):

Backend	Notes
Memory	Nothing persisted. Tests and throwaway agents.
SQLite	One file. Keeps only the working set in memory: active transcripts, live tasks, pending submissions. The choice for long-running agents.
JSONL	Append-only files in a directory. Keeps the whole history in memory.
Durable Object	The SQLite storage of one Cloudflare Durable Object.

The SQLite and JSONL cores use no Node APIs, so with a small adapter they run on Bun or inside a Durable Object. Because active transcripts are bounded by the model's context window (compaction sees to that), even a conversation with tens of thousands of messages keeps a small memory footprint on SQLite.

CHAPTER 7

Tasks: work as durable state machines

Everything the harness runs is a task: each request to the model, each tool call, each compaction. Extensions bring their own. A task is a small state machine whose current state is a row in storage, so a process that has never seen it before can pick it up.

The shape of a task

A task definition has a name and a version, a set of named **phases**, and an **abort handler**. A task record stores its input and a **checkpoint**: which phase is next, plus whatever that phase needs. A phase runs some code and ends by committing one of three things:

- a new checkpoint ("next, do phase B with this data"),
- a wait ("resume at phase C once these other tasks are done"), or
- an outcome (completed, failed, aborted).

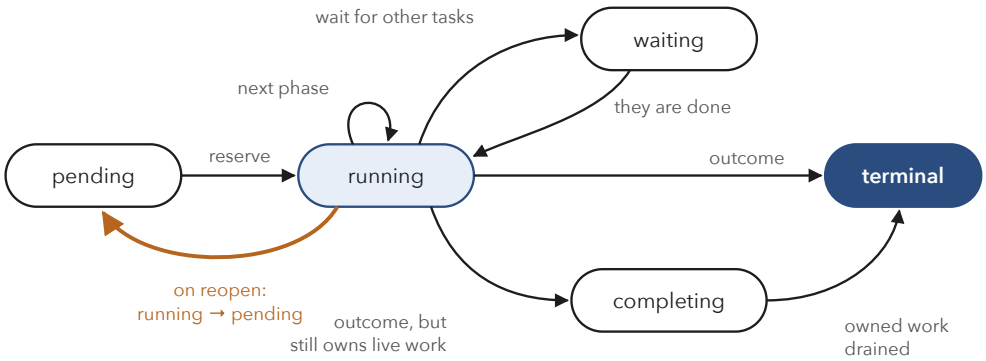


Figure 7.1 · Task states. Only "running" has code executing. On reopen, running tasks fall back to pending and are simply scheduled again.

Two scheduler rules make this robust:

- **A phase must make durable progress.** If a phase returns without having changed its checkpoint or committed an outcome, the task is *faulted*. Progress that is not written down does not count.

- **On reopen, running becomes pending.** Waiting tasks stay waiting. Nothing else happens at open; the scheduler later reserves pending tasks like any others and runs the phase in their checkpoint. There is no separate recovery code path per feature.

The effect sandwich

This is the most important idea in the task model, and the most honest one. Durability cannot make an external effect happen exactly once: if the process dies while a card is being charged, nobody knows whether the bank saw the request. What durability can do is make that uncertainty **explicit and local**:

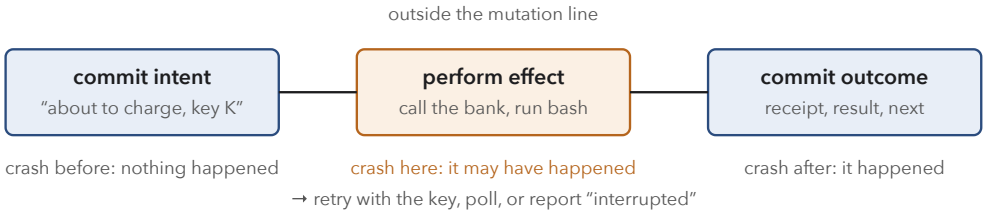


Figure 7.2 · Intent, effect, outcome. A restart that finds a task in its intent phase knows exactly which effect is in doubt.

A task that restarts in the middle knows precisely which effect is in doubt, and decides: retry safely with an idempotency key, poll an external handle ("is job 42 done?"), or report the interruption to whoever can decide. Tool calls use exactly this (chapter 8): a tool marked *replay safe* is simply run again; any other tool is reported to the model as interrupted, never repeated.

Memos: decisions you only make once

Sometimes a step has to decide something that must not be decided twice: an approval asked in Slack, a random choice, a generated name. A **memo** is a small value stored with the task where the first write wins. A hook that asks a human for approval stores the answer in a memo; after a restart it finds the stored answer instead of asking again.

Time, versions, and missing code

- **Timers survive restarts.** A task sleeps until a wall-clock time stored in its checkpoint. A retry backoff is "retry at 14:02:10", not "in five seconds". A reminder that

fires tomorrow is a task asleep until tomorrow.

- **Tasks are versioned.** A new definition can migrate old checkpoints when it reserves them.
- **Missing code blocks, never destroys.** If a task's extension is not installed (yet), the task stays pending and is reconsidered when the registry changes. The harness never marks work failed just because the code to run it is temporarily missing.

FOR THE TALK

"Durability can't make a side effect happen exactly once. What it can do is tell you, after a crash, exactly which effect is in doubt."

CHAPTER 8

One turn, taken apart

With the vocabulary in place, here is what actually happens when you ask an agent something. One answered question is a small tree of tasks, and every arrow below is a commit.

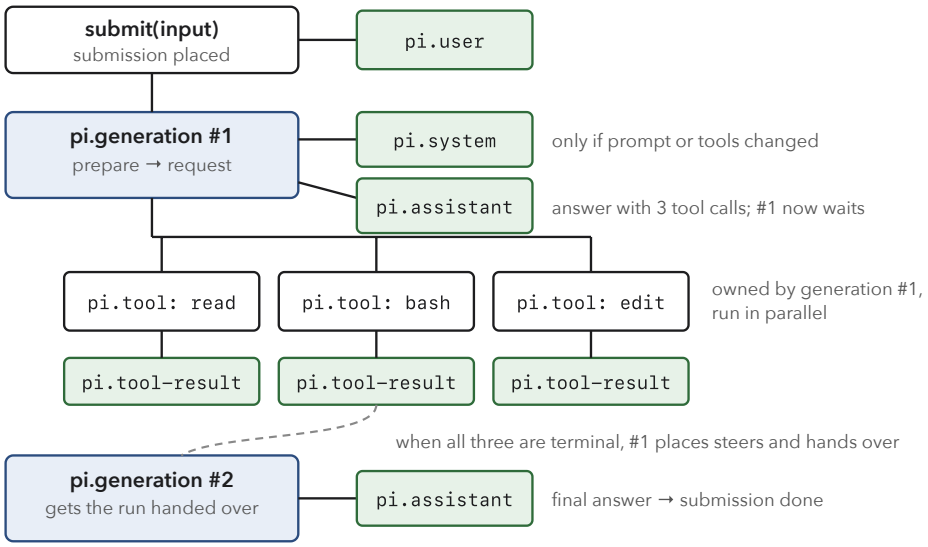


Figure 8.1 · One answered input. Blue: model-request tasks. White: tool tasks. Green: entries appended to the transcript.

Step by step

1. **Admission.** `submit()` durably records the input. If the conversation is idle it is placed at once as a `pi.user` entry, and a generation task is created. `pi.live.run` now says the conversation is busy and which task holds the run.
2. **Prepare.** The generation resolves the conversation's agent, renders the system prompt sections, and compares them with what the transcript says the model last saw. If anything changed, it appends a `pi.system` entry with only the difference. It checks whether the context is getting too long (chapter 12), then commits its next checkpoint: *request*, pinned to a model, a thinking level, and the last entry to include.

3. **Request.** The model streams. Every 100 ms or so, the partial answer is committed to `pi.live`. That is how every attached UI sees tokens appear.
4. **Tool round.** The response asks for three tools. One commit appends the answer, creates three `pi.tool` tasks owned by the generation, and puts the generation into *waiting* on them.
5. **Each tool.** A tool task validates the arguments and runs the `beforeTool` hooks (which may rewrite or block the call). Then it commits its *intent*: the final arguments and whether the tool is replay safe. Then it executes, streaming output into `pi.live`. Then it commits the `pi.tool-result` entry and finishes.
6. **Boundary.** When all three are terminal, the generation wakes up in its *tools* phase. This is a turn boundary: queued steering messages are placed here. It creates the next generation and hands it the run.
7. **Answer.** The second generation's answer has no tool calls. One commit appends it, marks the submission done, and clears the run. The conversation is idle.

Where can it die, and what happens then?

The process dies…	On reopen
after `submit()` returned	The input is stored. It is placed and answered as usual. A client that re-submits with the same `requestId` gets the same submission back, not a second question.
while the model streams	The committed partial is kept in the transcript as an *aborted* answer (visible, never sent to the model again). The same request is sent again, with the same pinned model and messages.
during a retry backoff	The generation sleeps until the stored deadline, then retries.
while a tool runs	Replay-safe tool: it runs again with the stored arguments. Otherwise: an *interrupted* error result, with the output stored so far, goes to the model, which decides what to do.
while the generation waits	It is still waiting; it resumes when its tools are done. Nothing to do.
in a subagent	The subagent is a conversation like any other; its own tasks resume (chapter 10).

WHY ONE TASK PER REQUEST

Splitting a run into a fresh generation task per model request, and a task per tool call, is what makes all of the above uniform. Each piece has its own checkpoint, so recovery is always local: continue *this* request, rerun *this* tool. The run itself is just a pointer in `pi.live`, handed from one generation to the next.

CHAPTER 9

One ownership tree

Durability answers “what happens after a crash”. Ownership answers the other hard question of long-running agents: “what happens when someone presses Esc?” In Pi Durable, all tasks and conversations form one tree, and that tree decides what an abort reaches and when something is finished.

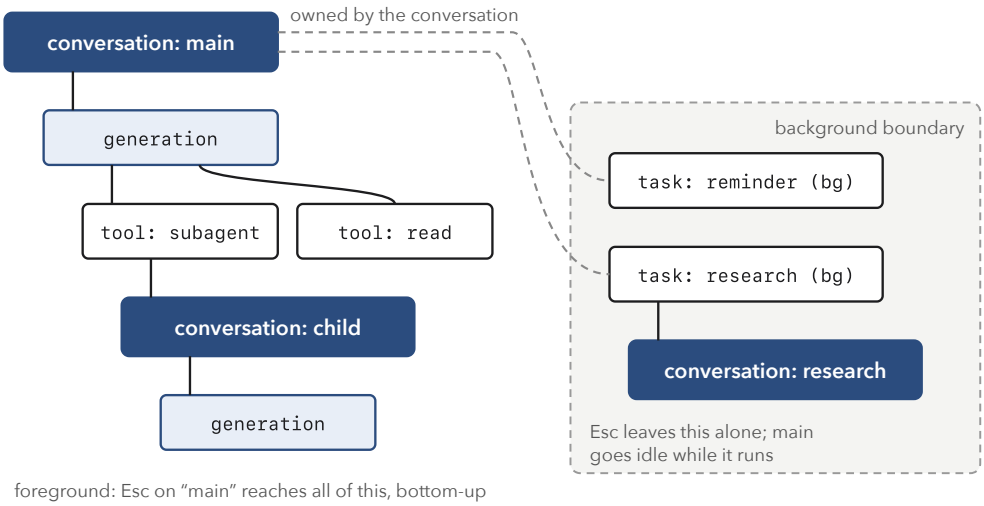


Figure 9.1 · Tasks own tasks, tasks own conversations, conversations own tasks. A background task is a boundary.

The rules

- **Everything names its owner.** A task is owned by its conversation or by another task. A conversation is ownerless or owned by a task. A subagent is simply a conversation owned by the tool call that started it.
- **Abort flows down.** Aborting something aborts what it owns.
- **Cleanup runs bottom-up.** A task’s abort handler starts only after everything it owns has finished. So each task cleans up its own effects, and a parent’s handler sees the final outcome of its children.
- **Owned work does not outlive its owner’s finish.** A task that decides its outcome while its children still run is held as *completing* until they drain. When a tool call re-

turns, its subagent is really done.

- **Background is a boundary.** A background task belongs to the conversation but not to its current work. The conversation goes idle while it runs, and Esc leaves it alone. Aborting the task itself, or the conversation with `{ background: true }`, still stops it.

Waiting together

A task can create children and then wait for them without running any code. With *allSettled* it resumes when all are done. With *failFast*, the first failure aborts the others. Chapter 19 uses this for a checkout that charges several cards at once and refunds them all if one is declined.

Aborted versus orphaned

There is a fine but important distinction in the outcomes. *Aborted* means the task's own abort handler ran and cleaned up. *Orphaned* means its code was not available (its extension was uninstalled, say) and someone aborted it anyway: no cleanup ran, and effects it started may remain. The harness never pretends cleanup happened.

FOR THE TALK

"Durability is the easy half. The hard half is Esc. When you press it, what stops? In Pi Durable, the answer is a tree, and cleanup runs from the leaves up."

CHAPTER 10

Many conversations: forks and subagents

One harness runs as many conversations as you need, concurrently, all with the same guarantees. New conversations start fresh, fork an existing one, or are created by a tool as a subagent.

Forks share history without copying it

A fork names a parent conversation and an entry in it. It sees the parent's transcript up to that entry, then continues on its own. Nothing is copied; reading a fork's history walks its own entries, then the parent's up to the cut.

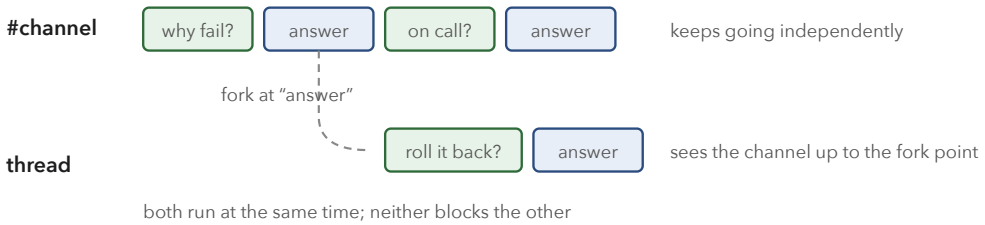


Figure 10.1 · The Slack example from the launch post: a channel is one conversation, a thread is a fork of it at the message it replies to.

Each conversation has its own agent

The `pi.agent` document stores, per conversation, what it runs with: model, thinking level, the selected extensions and which of their tools are active, extra instructions, and a working directory. A reviewer next to the main agent can use a cheaper model, read-only tools, and its own checkout. The Slack thread may search issues but not deploy.

Because the agent is a *rewindable* document, a fork starts with the agent its parent had *at the fork point*, not now. A subagent starts as a copy of its owner's agent at creation, and is then adjusted.

A subagent is a few lines, not a feature

Pi Durable has no built-in subagents, and that is the point: the primitives make them trivial. A subagent tool, described in prose:

1. In one commit: look for a conversation already owned by this tool call; if there is none, create one owned by it, and configure it with a smaller model and its own instructions.
2. Tell the UI which conversation it is, so the UI can show it under the call.
3. Submit the task to the child with a request ID derived from the tool call (`triage: <task id>`), and wait for the answer.
4. Return the answer as the tool result.

Now look at what that buys, without writing anything else:

- **Crash:** the tool is replay safe. A rerun finds the same child (step 1 looks it up first) and the same submission (same request ID), and keeps waiting for the same answer. The child's own tasks resumed on their own.
- **Esc:** the child is owned by the call, so aborting the parent aborts the child, bottom-up.
- **Idle:** the parent is not idle until the child is.
- **Cost:** the child counts its own usage.
- **UI:** the child is a conversation like any other. You can watch it, and you can steer it.

The *background* variant is a subagent owned by a background task: the tool returns immediately, the main agent keeps chatting, and a small task delivers the child's report back to the main conversation as a follow-up message when it arrives. The vacation planner (chapter 18) is built exactly like this.

CHAPTER 11

The inbox: many people, one agent

In the terminal you rarely type while the agent works, and if you do, it is you. In a Slack channel, anyone can mention the agent at any time, and the person who asked the first question may want to correct it mid-flight.

A conversation is **busy** while a run is working on an input. Anything submitted to a busy conversation goes into its durable inbox (`pi.inbox`) and is placed at a **turn boundary**, where it is safe to add to the transcript (never between a tool call and its result). Each submission says how it wants to be treated:

Mode	Behavior
steer	Placed after the current tool round and joins the running work. "Check the staging logs first."
follow-up	The default. Placed when the run answers, and starts the next run. "Also run the tests."
reject	Only if idle; otherwise the submit fails.
write	Appends an entry without asking the model anything. A display-only write (a chat message, "user opened a file") is appended at once, even mid-run.

Because the inbox is a document, every attached UI shows the queue, a queued message can be withdrawn, and a crash loses nothing that was queued. Because submissions take a `requestId`, a Slack webhook that is delivered twice asks the agent once.

An input can also be placed as an application entry: the model still sees "ann: can you check the build?" as a user message, but the transcript records a `chat.mention` with the author, which is what a UI wants to render.

THIS IS THE MULTIPLAYER MECHANISM

"Multiple humans steer the same agents" is not a separate feature. It is the inbox, plus the fact that every client reads the same committed state (chapter 14).

CHAPTER 12

Context without end

A conversation that lasts weeks will not fit in any context window. Pi Durable keeps everything in storage forever and changes only what the model sees, without stopping the agent to do it.

Heads: where the model's view begins

The transcript only ever grows. Some entries carry a **head**: a marker that says "from here on, the model's context starts at entry N". The *active transcript* is everything from the newest head to the tail. Older entries are still in storage and still searchable, just no longer sent.

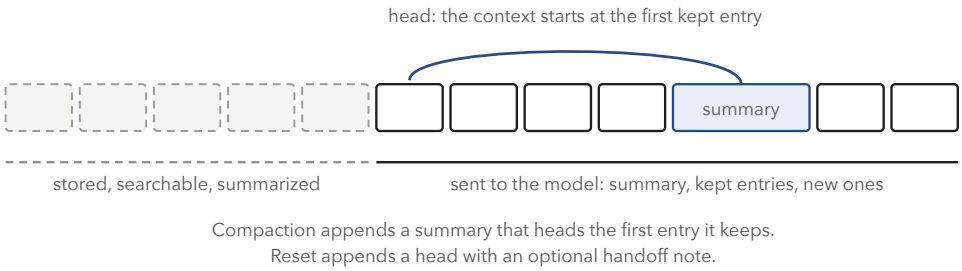


Figure 12.1 · Nothing is deleted; a head moves the window.

Compaction is a task that runs while the agent works

In the terminal agent, compaction is something the agent stops to do. In Pi Durable, compaction is a task like any other, and it runs *concurrently*:

- When the context gets within a configurable distance of the limit, a **background compaction** starts summarizing older messages. The agent keeps working.
- The summary is placed at the next turn boundary, like a queued message.
- The conversation only *waits* for a summary when the next request would not fit otherwise.
- If the provider still rejects a request as too long, the harness compacts and retries once.

- You can compact manually at any time, with your own instructions, also while the agent works.

Reset and handoff

`reset()` goes further: it starts a new context, optionally from a handoff note. A tool can request the same by returning a handoff. Because nothing is deleted, a second tool can still search everything before the handoff. That is all it takes to build an agent that hands off to itself and looks things up later.

The system prompt is part of the transcript

One more subtle piece. The system prompt is rebuilt from the selected extensions' sections before every request, so a changed section (an updated `AGENTS.md`, a new todo list) is picked up by the next request. But what the model *saw* is recorded in the transcript as positional `pi.system` entries holding only the differences: "section cwd changed", "tool read removed". Two things follow:

- A restart or a fork sees exactly what the model saw, at the position where it changed.
- On models that support it, only the change is sent, so the provider's prompt cache stays valid.

CHAPTER 13

Extensions: names, not code

Pi is malleable, so Pi Durable has to be. The trick that makes it possible for long-running agents is small: conversations store extension and tool *names*, never code.

Extensions and the registry

An **extension** is a named bundle of system prompt sections, tools, hooks, tool wrappers, and task definitions. The application installs extensions in a **registry** that the process owns. By default every conversation selects every installed extension; each can narrow that.

- A later extension's tool with the same name **replaces** an earlier one (a `bash` that runs inside a virtualenv).
- A **wrap** decorates whichever tool won (time every `bash` call, whichever `bash` it is).
- **Hooks** step into the built-in tasks: rewrite a request before it goes to the model, block or rewrite a tool call, replace a result, keep a run going, write a compaction summary yourself. Several extensions can hook the same point; their hooks run as a chain in selection order.

Hot reload, for free

The registry can change while conversations run. Installing an extension under a name that is already installed replaces it in one step:

- A tool call that is already running finishes on the code it started with. The next call uses the new code.
- A task picks up new code at its next phase boundary.
- After a restart, conversations pick up whatever the new process installs, because they only stored names.
- A stored name that is not installed right now is skipped, not an error. When it is installed again, it takes effect again.

This is the same mechanism that makes recovery work: the durable state never contains anything a different version of the code cannot read.

Hooks have to be durable too

A hook can run again after a crash. So a hook that makes a decision stores it in a memo. The canonical example: a `beforeTool` hook that asks for approval in Slack before any `deploy`. The answer is a memo on the tool task. After a restart, the hook finds the stored answer instead of asking again.

Tools calling tools, and durable code mode

A tool can call another tool. The nested call is its own task, owned by the caller, with its own hooks and replay policy. This is the basis for **code mode**: the model writes a small JavaScript program that calls the conversation's tools, run in a QuickJS sandbox. The clever part is how a script survives a restart:

- The sandbox is deterministic: a virtual clock, seeded randomness, UTC.
- Every result a script receives is written to a journal before the script gets it.
- After a restart, the script runs again from the top; it is handed the journaled results in the same order, so it makes the same calls with the same arguments, which reattach to the nested calls already made. Then it continues where it was.
- If the script takes a different path than before, or the sandbox changed, the call ends with an error instead of guessing.

This is the effect sandwich again, applied to a program instead of a single call.

CHAPTER 14

Documents, views and multiplayer

An agent is rarely the whole application. There is a todo list, a plan, a ticket, a shopping cart, the sandbox the conversation runs in. In Pi Durable this state is as durable as the conversation, and every client sees it the same way.

Your state, in the same commit

Application state lives in **documents**: typed JSON stored next to the transcript and changed in the same atomic commits. A `todo` tool that adds an item appends its tool result and changes the todo document in one commit, so the state never disagrees with the transcript that produced it. A prompt section can render the document so the model sees the list before every request; a UI subscribes to the committed value.

Each document says what a fork starts with, and this is a product decision:

Fork mode	A fork starts with…	Example
`asOf`	the parent's value at the fork point	todos: the thread sees the list as it was then
`current`	the parent's value now	a container assignment
`initial`	a fresh value	a scratchpad, the provider session ID

Views: snapshot, then exact changes

Everything a UI needs is committed state. A client that attaches to a conversation gets the current **view** first: the active transcript, the answer being streamed, running tools and their output, queued messages, the agent, usage. After that it gets only what changes, as the exact JSON operations of each commit: small enough to send over a WebSocket.

- **Late join:** a client that joins mid-run, or reconnects, starts from the current view. Nothing is replayed.
- **Slow clients:** a watch buffers up to 100 commits. Past that, the backlog is replaced by one frame with the whole current view.

- **Familiar events:** if you prefer the coding agent's events (message start, message update, tool execution start…), `watchEvents()` derives them from the commits, at the cost of more bytes.
- **Task graph:** every live task of the session as one view, for a task panel or for debugging.

Combined with the inbox, this is multiplayer: any number of people and clients watch the same conversation, join late, and steer it.

WHY THIS FALLS OUT OF THE ONE RULE

Because nothing is shown before it is stored, a "live view" and a "view after a restart" are the same thing. There is no event stream that has to be kept consistent with the database: the stream *is* the database's commits.

CHAPTER 15

Running anywhere

"Anywhere" currently means anywhere there is a JavaScript runtime. Two small interfaces make that possible: storage, and the execution environment.

The execution environment

Tools that need files or a shell get them from an **execution environment**, never directly from the machine the harness runs on. Pi Durable ships a Node environment for local files; the interface is small enough to implement for a remote VM or a container. An `env` function builds the environment for every tool call, from the conversation's working directory and its documents. So:

- the harness can run on one machine while its tools run on another;
- each conversation can run in a different place: a directory per conversation, or a container per conversation recorded in an application document;
- a tool rerun after a crash gets the conversation's environment *at that time*.

Example: an agent in a Durable Object

There is an example project (`pi-durable-cloudflare`) that hosts a harness inside a Cloudflare Durable Object. It is a good illustration of how little glue "anywhere" needs:

- Each named agent session is one Durable Object; the harness state is its SQLite.
- The browser talks to it over one WebSocket and renders the event stream.
- A `schedule` tool creates a durable task that delivers an instruction back to the same conversation when it is due. The earliest deadline becomes the Durable Object's alarm.
- When Cloudflare evicts and later reconstructs the object, the alarm wakes it, the harness reopens, and every task continues from its checkpoint.
- A `recall` tool is a foreground subagent that searches other conversations and answers briefly, so raw matches never enter the parent's context.

Eviction, hibernation, redeploys: to Pi Durable these are all the same event as a crash, and handled the same way.

CHAPTER 16

What can go wrong, and what is not promised

A durable system is defined as much by what it refuses to promise as by what it does. This chapter is useful for the Q&A.

Three rules decide what an error does

1. **A storage error ends the harness.** If storage fails, nothing can be recorded, so nothing may continue. The harness fails, closes itself, and must be reopened. Commits are atomic, so the store is consistent; resubmitting with the same request ID finds what was already admitted. Storage retries its own transient failures.
2. **An invalid request fails only that call.** A bad cursor, an unknown conversation.
3. **Your code fails only its own unit.** A throwing task phase faults that task. A throwing tool gives the model an error result. A throwing hook is reported and skipped, except `beforeTool`, where a throw blocks the call (failing closed).

What Pi Durable does not do

Not promised	Instead
Exactly-once external effects	The effect sandwich makes the doubt explicit; idempotency keys, polling, or "interrupted" resolve it.
Several processes writing one storage	One owner at a time, no cross-process lock. Other clients attach to the owner.
Offline multi-writer merge (CRDTs)	One line of commits; documents are changed by one writer.
Killing misbehaving code in-process	Code that ignores its abort signal delays close until it returns.
Power-loss durability on JSONL by default	Opt into fsync, or use SQLite (WAL; the newest commit may be lost on power failure).
A stable API	It is experimental. The API changes between releases.

Footguns worth knowing

- **Long transactions.** Never await a model, a process or a human inside a commit. It holds the line for everyone.
- **Owned work holds its owner.** A subagent you keep talking to keeps the tool call that owns it in *completing*. Work that should not hold its owner is created as background.
- **Unstable prompt text.** A section that embeds the current time changes on every request, appends a system delta every time, and defeats prompt caching.
- **Guards are selected, not global.** A conversation whose extension selection omits the permissions extension runs without it.

PART III

Seeing it work

Three scenarios in slow motion. Each is told twice: what a user sees, and what is in storage. The second telling is the material for a demo that explains instead of just showing.

CHAPTER 17

Pulling the plug: a crash, commit by commit

A Slack bot built on Pi Durable gets a message. Halfway through the work, its container is redeployed. This chapter follows every commit, then the restart, then the end of the story.

The setup

Someone writes in a channel: *"@agent fix the flaky login test"*. The Slack integration submits it with the Slack event ID as `requestId`. The agent has two relevant tools: `search_issues`, which only reads and is marked replay safe, and `bash`, which is not (running a command twice is not something the harness will decide on its own).

Act one: the work

#	What happens	What the commit writes
1	Message admitted	Submission S1 (requestId `Ev042`). Entry 1 `pi.user`. Task T1 `pi.generation`, pending. `pi.live.run` = T1 answering S1.
2	Scheduler picks T1	T1 pending → running.
3	T1 prepares	Entry 2 `pi.system` (prompt sections and tools). T1 checkpoint → *request*, model pinned, cutoff entry 2.
4–9	Model streams	`pi.live.generation` partial, every ~100 ms. Every attached UI shows tokens.
10	Model asks for two tools	Entry 3 `pi.assistant` with two calls. Tasks T2 (`search_issues`) and T3 (`bash: npm test`), owned by T1. T1 → waiting on T2, T3. `pi.live.tools` slots.
11	T2 records intent	T2 checkpoint → *execute*, arguments, replay safe. Slot running.
12	T3 records intent	T3 checkpoint → *execute*, `npm test`, replay unsafe. Slot running.
13–20	Tests print output	T3's slot output: "running 214 tests… 120 passed…"
21	Search finishes	Entry 4 `pi.tool-result`. T2 terminal, completed.

22	Second person replies, as a steer: *"check the staging logs first"*	Submission S2, mode steer, queued in `pi.inbox`. (The run is busy; it waits for a boundary.)

KILL -9

The container is redeployed. The process is gone, mid `npm test`. The Slack integration never got an answer, and its webhook will be delivered again.

What is on disk at the moment of the crash

Entries

```
1 pi.user fix the flaky login test
2 pi.system sections, tools
3 pi.assistant calls: search, bash
4 pi.tool-result search: 3 issues
```

Tasks

```
T1 generation waiting on T2,T3
T2 tool search terminal
T3 tool bash running · execute
npm test, unsafe
```

Submissions

```
S1 input placed (Ev042)
S2 steer queued
```

Documents

```
pi.live.run T1, S1
pi.live.tools bash: "120 passed…"
pi.inbox [S2 steer]
pi.usage 1 request
```

Figure 17.1 · Everything a new process needs. Note what is not there: no recovery log. This is simply the current state.

Act two: a new process

#	What happens	What the commit writes
R1	Harness opens	Reconciliation: T3 running → pending. T1 stays waiting. Nothing runs yet.
R2	Scheduling starts; T3 reserved	T3 → running, in phase *execute*. This phase is only ever reached by recovery.
R3	T3 decides	`bash` is not replay safe, so it does not run again. Entry 5 `pi.tool-result`, error: "Tool bash was interrupted and may have partially run", with the output stored so far. T3 terminal, failed.
R4	T1 wakes, turn boundary	Steer S2 placed as entry 6 `pi.user`. Task T4, the next generation, gets the run.

R5…	T4 requests	The model sees: the request, its own tool calls, the search result, the interrupted test run with its partial output, and the steer. It decides what to do: read the staging logs, then rerun the tests.
…	Webhook redelivered	Submit with requestId `Ev042` returns S1. No second question.
end	Final answer	Entry `pi.assistant`. S1 done. `pi.live.run` cleared. Slack gets one answer.

What to notice

- **There was no recovery code.** Reopen moved one row from running to pending. Everything else is the ordinary machinery: a task in a phase, a boundary, a new generation.
- **The harness did not guess.** It did not rerun `npm test` on its own, and it did not pretend the test run finished. It told the model exactly what is known.
- **The second person was not lost.** The steer was durable before the crash and placed at the first boundary after it.
- **The search was not repeated.** It had finished; its result was in the transcript. Had it been interrupted too, it would simply have run again, because it is marked safe.
- **Slack sees one answer.** The request ID made the retried webhook a no-op.

FOR THE TALK

This chapter is the best candidate for the demo. Its shape is: show the work as rows appearing, pull the plug, show that the rows are still there, start a new process, and show that exactly one row changes before work continues.

CHAPTER 18

The vacation planner, underneath

The launch post showed a vacation planner: about 1,300 lines of TypeScript, most of them a TUI borrowed from the coding agent. Its storyline has eight beats. Here is each, with what is happening underneath.

#	What you see	Underneath
1	"Plan a weekend in Vienna for two. Hand the research to a subagent."	An ordinary submission and generation. The model calls the `research` tool.
2	A subagent runs three searches in parallel: weather, museums, trains. Each shows in the task panel.	`research` creates a *background* task owned by the main conversation; that task owns a new subagent conversation and submits the request to it with request ID `research:<task>`. The tool returns at once. The subagent's model calls `search` three times: three `pi.tool` tasks owned by its generation. The searches are canned and take 6, 10 and 30 seconds.
3	Meanwhile, the main agent is free to chat.	The research task is background: the main conversation is idle, so new input starts a new run there. The two conversations do not block each other.
4	The process dies. Weather and museums are done, trains is not.	On disk: two tool tasks terminal with results, one running in *execute*. The subagent's generation waiting. The research task waiting for the subagent's answer.
5	Restart with `--continue`. Only trains runs again.	Trains: running → pending → reserved in *execute*. `search` is replay safe, so it executes again with the stored arguments. Weather and museums are untouched.
6	Switch to the subagent and steer it.	The subagent is a conversation like any other. A steer goes into its inbox and is placed at its next boundary.
7	Back in main: ask, compact, and steer, all while it works.	A manual compaction is a task in the main conversation; it summarizes while the run continues, and its summary is placed at the next boundary.
8	The report arrives as a message; main turns it into a plan.	The research task reads the subagent's answer and submits it to the main conversation as a follow-up, with its own request ID, so a restart never posts the report twice.

Two things make this story good for a talk. First, the crash is natural: one search is slow on purpose, so there is a moment where some work is done and some is not. Second, everything in it is a conversation or a task, so the task panel shows the archi-

tecture directly.

AVAILABILITY

The vacation planner and the small durable coding agent lived in `packages/coding-agent/src/experimental`. They are in the tags v1.0.0 through v1.1.0, and were removed from main on 12 October (commit `7472cda0d`, together with the experimental server stack). v1.2.0 does not include them, and the launch post's links to them now point at nothing on main. To record them, check out v1.1.0.

CHAPTER 19

Not just for models: a split payment

The task machinery is not specific to language models. The launch post makes that point with a checkout that splits the bill across several cards. It is the clearest illustration of ownership, waiting and bottom-up cleanup.

Two task types

- **Payment** has one phase, *charge*: charge one card with the idempotency key `payment-<task id>`, then commit the receipt or the decline as its outcome. Its abort handler refunds by the same key.
- **Checkout** has two phases. *pay* creates one Payment per card, owned by the checkout, and commits "wait for all of them, fail fast". *decide* reads their outcomes and commits "order placed" or "a payment failed".

An agent starts a checkout with a tool. The checkout is owned by that tool call.

Four things that can happen

Scenario	What the tree does
All four cards go through	Each payment completes. The checkout, which ran no code while it waited, resumes in *decide*: placed.
One card is declined	That payment fails. *failFast* marks the others that are still in flight for abort (here: all three). Each runs its own abort handler: a refund. When all four are terminal, the checkout resumes and reads `[aborted, failed, aborted, aborted]`.
The user presses Esc	Abort flows down from the conversation, through the generation and the tool call, to the checkout, to the payments. Cleanup runs from the bottom: payments refund first, then the checkout's abort handler sees their final outcomes.
The process dies mid-payment	The payments still in *charge* become pending and charge again, with the same key, so the bank charges once. The checkout is still waiting and resumes when they are done.

Notice that "refund on cancel" was written once, in the payment's abort handler, at the level that did the effect. Nobody above it needs to know how to undo a charge.

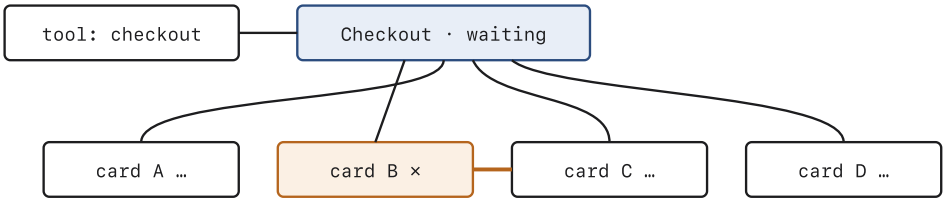


Figure 19.1 · A checkout that owns four payments.

A SUBTLETY WORTH KNOWING FOR Q&A

failFast aborts only the payments that are still live. A payment that already *completed* before another was declined is not aborted, so its abort handler does not refund it. In the example in `test/examples/24-child-tasks.ts` the expired card is declined at once, before any other charge completes, so all three others are refunded. A production checkout would refund completed payments in its *decide* phase. The machinery gives you the outcomes; the compensation policy is still yours.

PART IV

Toward the talk

Ten minutes, one idea, one demo. These chapters are a starting point for a discussion, not a finished plan.

CHAPTER 20

How to demo something you should not demo live

The thing Pi Durable is good at is invisible when it works. A durable agent that answers a question looks exactly like a non-durable one. The interesting moment is a failure, and failures on stage are either fake or risky. So the demo has to make the invisible visible.

What the demo must show

1. Work in progress is stored, not held in memory.
2. The process dies, and the state does not.
3. A new process continues, and changes almost nothing to do so.
4. Effects in doubt are handled honestly: rerun if safe, otherwise reported.

A terminal demo shows 2 and 3 but not 1 or 4: you see the agent stop and start again, and you have to take the rest on faith. That is the argument for explaining the architecture rather than just running it.

Options

Option	What it is	Trade-offs
A. The X-ray (recommended)	A split slide. Left: a chat, as a user sees it. Right: the storage, as four small tables and the `pi.live` document. Each click is one commit; rows appear on the right while text appears on the left. Then the left side goes dark ("kill -9"), the right stays. A new process appears, one row changes from running to pending, and work continues.	Zero risk; explains all four points; works at any speed. Needs building as an animated slide. Risk of feeling like a diagram, so it has to move.
A+. Recorded truth	Same as A, but the data is real: run a real harness (faux model provider, SQLite) through the chapter 17 scenario, record every commit with the commit subscription or a watch, kill it, reopen, record again. The slide replays the recorded trace.	Everything on screen is genuinely what the system wrote. A few hours of work; makes a great "this is not a mockup" line.

B. Screencast	A 30–45 second recording of the vacation planner: three searches, Ctrl+C with only trains left, `--continue`, only trains reruns, the report arrives. Sped up, with captions.	Proof that it is real, with a real TUI and task panel. Must be recorded from v1.1.0 (the demo is gone on main). On its own it shows the effect, not the mechanism.
C. Live, deterministic	Run an example like `13-recovery` or `24-child-tasks` live in a terminal. They use no network and print what happens.	Real and safe, but terminal text is not a good visual for an audience, and it costs time.
D. Live kill	Run the planner or a Slack bot live, kill it, restart it.	The most dramatic, and the most fragile: network, model latency, Wi-Fi. Not compatible with a ten-minute slot.
E. Two screens	Two browser tabs on the same conversation (the Cloudflare example): type in one, watch it in the other; reload one mid-answer; it comes back where it was.	Shows multiplayer and late join beautifully. Also live and network dependent.

A suggestion

Make A (ideally A+) the spine of the talk, and do not call it a demo: call it *following one message through the system*. Then, optionally, close the section with ten seconds of B as the punchline ("and here is the real thing doing it"). That gives you the explanation and the proof without anything that can fail on stage.

Sketch of the X-ray, beat by beat

Beat	Left (what the user sees)	Right (what storage holds)
1	"@agent fix the flaky login test"	A submission row; an entry row; a task row, *pending*.
2	The answer streams in	`pi.live` partial text growing, in sync. Make the point: the left is drawn from the right.
3	Two tool calls appear, one finishes	Two task rows (*search: safe*, *bash: unsafe*), each showing its intent checkpoint. Search goes terminal; its result entry appears.
4	A second person writes "check the staging logs first"	An inbox item appears, queued.
5	Darkness. "kill -9"	Unchanged. Pause here. This is the slide.
6	A new process	One row changes: bash *running* → *pending*. Nothing else.

7	"bash was interrupted", then the steer, then work resumes	The interrupted result entry; the steer placed; a new generation row.
8	One answer in Slack	The submission goes *done*. A duplicate webhook bounces off the request ID.

Optional ninth beat if there is time: press Esc on a run with a subagent, and show the ownership tree aborting from the leaves up. It demonstrates the other half (chapter 9), but ten minutes may not allow both.

CHAPTER 21

A ten-minute shape

One possible structure. The times add up to ten minutes; the slide counts are rough.

Time	Section	Content
0:00–0:45	Hook	"What happens when your agent dies?" In a terminal: you notice and type "continue". You are the recovery mechanism.
0:45–2:30	Why	Agents are leaving the terminal: Slack, email, GitHub, Durable Objects. Nobody watches the process. The five requirements (run anywhere, many surfaces, endless conversations, survive crashes, many humans), plus malleability. "If the human is not the recovery mechanism, the harness has to be."
2:30–3:30	How we got here	First design: the transcript plus recovery notes in the margin. What shipped: a small database where the transcript is one table and every piece of work is a row that knows where it was. Recovery stopped being a feature and became a consequence.
3:30–6:30	Four ideas	**1. Commit before you show.** One line of commits; the screen is a window onto storage. **2. Everything is a task.** Model requests, tool calls, compaction. Intent, effect, outcome. Durability can't make an effect happen once; it can tell you which one is in doubt. **3. One ownership tree.** Subagents are conversations owned by tool calls. Esc flows down, cleanup runs up. Background is a boundary. **4. Names, not code.** Conversations store extension names; code comes from the registry. Hot reload and restarts are the same thing.
6:30–9:00	Follow one message	The X-ray (chapter 20): eight beats, the plug pulled at beat five. Optionally ten seconds of the vacation planner as proof.
9:00–10:00	Close	Experimental, try it, point your agent at `packages/durable`. What we are building with it. The coding agent is not going away; lessons flow back.

Lines that might be worth keeping

- "A coding agent can rely on you to notice that it died. A Slack bot can't."

- “Nothing is shown before it is stored.”
- “The screen is a window onto the database, not onto the process.”
- “Durability can’t make a side effect happen exactly once. It can tell you exactly which one is in doubt.”
- “Recovery stopped being a feature and became a consequence.”
- “Durability is the easy half. The hard half is Esc.”
- “A subagent is not a feature. It is a conversation owned by a tool call.”
- “Conversations remember names, not code. That is why you can change the code.”

What to leave out

Documents and fork semantics, code mode’s deterministic replay, the details of compaction thresholds, storage backends beyond a single mention, hook chains, the task graph. Each is interesting; none survives a ten-minute cut. Keep them for questions. Code mode’s journal (chapter 13) is a particularly good answer to “what about long-running scripts?”

Likely questions

Question	Short answer
Isn’t this just Temporal for agents?	Same family of idea (durable execution), different trade-offs: no separate server, runs inside your process or a Durable Object, and the unit is a conversation with a transcript, not a workflow function. The transcript and app state live in the same commits as the tasks.
Exactly-once tool calls?	No. Intent is committed before the effect; after a crash, replay-safe tools rerun and others are reported as interrupted to the model. External effects need idempotency keys.
What does it cost?	A commit per visible change, throttled (100 ms for streaming by default). On local SQLite this is cheap; remote storage can commit less often.
Multiple servers?	One process owns a storage at a time. Scale by sharding sessions, as the Durable Object example does (one object per session).
Why TypeScript?	Easiest way to bootstrap; runs everywhere there is JavaScript. A port is not ruled out.
Will Pi the coding agent move to it?	Pi Durable does not replace it. Lessons flow back as they prove themselves.

CHAPTER 22

Facts, numbers, and things to double-check

Numbers that might go on a slide, with where they come from. Check them again close to the talk; the package moves fast.

Fact	Source / caveat
Shipped 1 October 2026, alongside Pi 1.0, as experimental	Launch post.
Package `@earendil-works/pi-durable`, current version 1.2.0	`package.json`; v1.2.0 tagged 13 October.
About 15,000 lines without tests at launch (≈150k GPT tokens, ≈250k Claude tokens)	Launch post. Today `src/` totals about 25,000 lines, of which about 2,900 are testing helpers (conformance suites), 3,200 storage, 1,700 code mode, 2,300 execution environments, 8,500 the harness itself. Recount before quoting.
Specification: about 5,300 lines	`docs/spec.md`.
32 runnable examples	`test/examples`, numbered 00 to 31. Most run without an API key using a faux provider.
Storage backends: memory, SQLite, JSONL, Cloudflare Durable Object	README.
Streaming partials and tool output committed at most every 100 ms	Default `progress` settings.
Watches buffer 100 commits before collapsing to a full view	README, spec section 12.
Compaction defaults: reserve 16,384 tokens; background starts 32,768 earlier; keep ~20,000 recent	Spec section 2.2.
Retry defaults: 3 retries, 2 s base delay, 60 s cap	Spec section 2.2.
First design document: 28 July 2026; package created 18 September	Git history. Pico prototypes in between.
Vacation planner: ≈1,300 lines, mostly TUI	Launch post; the commit adding it is 1,378 lines including README.

THINGS TO RESOLVE BEFORE THE TALK

- **Broken demo links.** The launch post links to the durable coding agent and the vacation planner under `packages/coding-agent/src/experimental`. Both were removed from main on 12 October and are not in v1.2.0. Either restore them, link to v1.1.0, or do not send the audience there.
- **Model names.** The post's examples use model IDs like `gpt-6.1-sol` and `gpt-6-luna`. Make sure whatever appears on slides is current.
- **Line counts.** Decide whether to quote the launch number (15k) or a fresh count.
- **The checkout example.** As written, a payment that completes before another is declined is not refunded (chapter 19). Avoid presenting it as complete compensation logic.

APPENDIX A

Glossary

Term	Meaning
Abort handler	The code a task runs once when aborted, after everything it owns has finished; decides the aborted outcome and cleans up its own effects.
Active transcript	The entries from the newest head to the tail; what the model sees.
Agent	What a conversation runs with: model, thinking level, extensions, tools, instructions, working directory. Stored as names in `pi.agent`.
Background task	A conversation-owned task outside the conversation's current work. Esc does not reach it; the conversation can be idle while it runs.
Boundary	A point between turns where queued submissions can safely be placed.
Checkpoint	A task's stored progress: the next phase and its data.
Commit	One atomic write of entries, documents, tasks and submissions.
Compaction	A task that summarizes older messages and places a headed summary entry.
Completing	A task that has decided its outcome but still owns live work; it becomes terminal once that drains.
Conversation	A transcript scope. May fork another at an entry; may be owned by a task.
Document	Typed, mutable JSON state committed with entries; scoped to session, conversation, or task.
Effect sandwich	Commit intent, perform the effect, commit the outcome.
Entry	An immutable transcript record.
Environment	Where tools touch files and processes; built per call.
Extension	A named bundle of sections, tools, hooks, wraps and tasks.
Fork	A conversation that sees its parent's history up to an entry, without copying.
Generation	The built-in task for one model request (`pi.generation`).
Head	A marker on an entry saying where the model's context starts.
Hook	Extension code that steps into a built-in task (before a request, before a tool…).
Inbox	The queue of submissions waiting for a boundary (`pi.inbox`).
Memo	A small first-writer-wins value stored with a task, for decisions made once.
Mutation line	The single serialized path all commits take.

Orphaned	An aborted task whose code was unavailable, so no cleanup ran.
Registry	The extensions this process installed; can change while running.
Replay safe	A tool that may simply run again after a crash interrupted it.
Request ID	A caller-chosen key that makes a submission exactly-once.
Run / turn	A turn is one model response and its tool calls; a run is the turns from an input to its answer.
Session	The transactional core under a harness: one storage, one mutation line.
Steer / follow-up	Input placed after the current tool round / after the run answers.
Submission	Input or a write handed to a conversation; queued, placed, done or unanswered.
Task	A durable state machine: input, checkpoint, phases, abort handler, owner.
View / watch	A conversation's committed state for UIs; a watch delivers each commit's exact operations.

APPENDIX B

A map of the source

If you, or your agent, want to read the code: where things live in `packages/durable/src`, with approximate line counts.

Path	Lines	What it is
`session/`	2,500	The transactional Session: commits, transactions, forks, document observation.
`harness/scheduler.ts`	1,950	Reservation, reconciliation, abort cascades, ownership traversal.
`harness/tool.ts`	900	The `pi.tool` task: validate, hook, intent, execute, replay.
`harness/generation.ts`	700	The `pi.generation` task: prepare, request, retry, poll, tools.
`harness/compaction.ts`	460	The `pi.compaction` task.
`harness/harness.ts`	470	Opening, conversations, the public surface.
`harness/` (rest)	4,000	Agent resolution, prompt, inbox, submissions, views, events, task graph, registry.
`storage/`	3,200	Memory, SQLite (Node and Cloudflare), JSONL.
`env/`	2,300	The execution environment interface and the Node environment.
`tools/`	1,550	read, write, edit, bash.
`codemode/`	1,700	Durable code mode: sandbox, journal, replay.
`testing/`	2,900	Storage and environment conformance suites, benchmarks.
`*.ts` (root)	2,000	Types, documents, entries, tasks, IDs, errors, truncation.

Recommended reading order for a human: the README's Concepts section; then `test/examples/06-harness.ts`, `13-recovery.ts`, `22-subagent-foreground.ts` and `24-child-tasks.ts`; then spec sections 1, 5.2 and 5.5.

End.